ARC-AGI Milestone B

Scott Schmidl
sschmidl3@gatech.edu

***Abstract*—**In this milestone, I discuss my approach to a rule-based ARC-AGI agent used to solve the Milestone B problems. The agent treats each problem as a search over a bank of grid transformations and builders meant to infer specific transformations from the training pairs. The main rule bank captures common transformations such as rotations, mirroring, and cropping, while the learned rules specialize to patterns such as marker shapes. For each problem, the agent filters potential rules by requiring an exact match on all training examples, then applies the rest of the transformations to the test grid, and returns up to three unique predictions. With this approach, I was able to achieve a strong baseline for future milestones with a 16/16 correct on the training set and 13/16 correct on the test set. I will analyze where the agent succeeds and fails, discuss runtime, and outline further additions to help generalize better and have success on Milestones C and D.

1 HOW DOES YOUR AGENT CURRENTLY FUNCTION?

For Milestone B, my agent takes a rule-based approach that searches a bank of transformations and rule-builders to explain the training pairs and then applies them to the test set. For each problem, the agent loads all training pairs and testing input data, then iterates over potential rules searching for those that can exactly reproduce every training output. The rules map input to output grids or nothing if no rule worked. The agent is essentially performing hypothesis testing over the bank.

The first group of hypothesis are the static rules. These include, a vertical mirroring rule that takes a bottom pattern and mirrors it upwards, a spiral pattern generator that fills an empty square grid with a colored spiral, a rule that convert solid colored rectangles into hollow rectangles, and a dilation rule that grows cells of color 5 into 3x3 blocks, and a rule that draws an "X" through a single colored pixel. Other rules handle simple color statistics by replacing the rarer of two colors with 2, masking, cropping around the non-zero region, swapping the two colors in a cropped region, and rotating the grid by 90 or 180 degrees.

The agent also has a set of learned rules that creates problem specific rules from the training pairs. These include the following:

- A color filter builder that looks for cases where all outputs are single-colors by taking one input color and changing it to a fixed output color then constructs that applies this to the test grid.
- An OR-panel which splits inputs into left and right panels and checks if the output equals a logical OR of the two non-zero masks. If it does, it creates a rule that reproduces this OR on the test grid.
- A role color which detects holes and borders to learn a pattern where a shape color gets outlined and filled with two new colors depending on border vs holes.

The main loop generates the potential functions in three passes:

- The static rules.
- The rules builders.
- Panel masks built from combinations of panels using logic gates.

Each rule is applied to every training input and checks for equality with the training output. If a rule doesn't pass all training pairs, it gets rejected. When predicting, the agent applies each remaining rule to the test grid, keeps only unique predictions, and then returns up to three distinct predictions.

2 HOW WELL DOES YOUR AGENT CURRENTLY PERFORM?

The agent performs strongly on Milestone B. My current approach solves 16/16 training problems and 13/16 testing problems, giving me confidence that my design is working as intended. Getting 16/16 on the training set suggests that the current bank of rules and builders is good enough to explain these examples. The drop in performance for the test set suggests that some of the learned rules are still too narrow, or that some transformations or compositions were missed in the current approach.

Overall, I think these current results are a solid baseline. It is not simply guessing, but instead is able to solve a wide variety of problems using a consistent process, i.e. propose rules, validate them, then apply them rules to test set.

3 WHAT PROBLEMS DOES YOUR AGENT PERFORM WELL ON?

The agent performs best where the correct transformation is a single, consistent rule applied across the entire grid. This includes problems involving symmetry, rotation, cropping, recoloring, masking, and geometric growth. These are all cases where input-output relationship is stable enough that one rule can explain all training pairs cleanly

It also does well on problems where color has a clear role. For example, if one color acts like a marker, mask, or border, the agent is often able to detect that role and generate the correct output. In such cases, the problem is less about raw pixel matching and more about recognizing that a specific color is functioning in predictable ways.

Another area of strength, is problems that have an object in them, especially when objects are clearly separated and their behavior is consistent. If the task is to isolate a non-zero region, transform it, or preserve one part of the grid while changing another in a consistent pattern the current approach does well. In general, the agent is successful when it has consistency and the transformations don't have many dependencies.

4 WHAT PROBLEMS DOES YOUR AGENT STRUGGLE ON?

The agent struggles more on problems where the correct solution is more complex than a single transformation, but instead depends on multiple steps or deeper understanding of the case. In those cases, even if part of the pattern is recognizable, the current design may not be able to combine the needed methods into a good hypothesis. A rule might explain one aspect of the training examples while missing another requirement that is important in the test case.

It also has trouble when the correct output depends on more relationships than a current rule supports. For example, if the task depends on comparing several objects or building a new rule from multiple interacting parts, the agent has fewer good candidates to test. Right now, the agent is stronger at recognizing regular transformations than at discovering a more abstract transformation behind the grid.

Another weakness is that the learned rules depend on specific assumptions. When those assumptions hold, the agent can generalize well. When they don't,

the builder fails and the problem falls back to the static rules. That means the agent is more reliable on problems with clear, repeated structure than those with irregular layouts, ambiguous object roles, or transformations that require broader context across the grid.

5 HOW EFFICIENT IS YOUR AGENT?

The current agent is reasonably efficient because it searches over a finite set of transformation instead of exploring a large open-ended search space. For each problem, it checks candidate rules against the training pairs and immediately reject any rule that fails to match outputs exactly. That makes the search process more direct, since the agent doesn't spend much time refining weak candidates once they fail the training data.

Runtime grows with the number of candidate rules being tested, the number of training pairs in the problem, and the size of the grid. Most of the rules are simple enough that they run quickly on input, so the overall runtime stays manageable. Problems with larger grids or more connected components do take longer, especially when the agent needs to make more decision about the objects in the grid, but even then the slowdown is moderate.

From a complexity perspective, the behavior is about linear, O(n), in the number of candidate rules and in the total amount of grid space being checked, with a bit of extra time for component detection or structural analysis. There hasn't been much of a bottleneck for Milestone B. The current design is relatively lightweight enough that I can iterate on rules quickly. The overall runtime of my design is 0:00:00.076045 or .08 seconds, which implies milliseconds per problem on average. Locally, generating the CSVs, the agent took about 0.028 seconds, suggesting no noticeable slowdown on particular problems.

6 HOW DO YOU PLAN TO IMPROVE YOUR AGENT'S PERFORMANCE ON THESE PROBLEMS BEFORE THE FINAL PROJECT SUBMISSION?

The best next step is to make the design less dependent on single transformations. Before the final project submission, I want to expand the rule mapping by allowing short compositions of rules so the agent can represent problems that require two related operations. This would make the system more flexible without abandoning the rule-based design that has worked on this milestone.

I also want to expand the set of rules that better captures common patterns, not just add more rules, but add better rules. More specifically, I want to improve how the agent handles selection of multiple objects, more complicated spatial relationships, and cases where the output depends on a combination of attributes such as color, position, and shape.

Finally, I want to improve the debugging process around failures. At this point, the most useful improvements will come from understanding why a rule(s) almost works but fails on one training pair or one hidden test case. Better logging and inspection of failed predictions should help me identify where the rule(s) is weak and needs improvement.

7 HOW DO YOU PLAN TO GENERALIZE YOUR AGENT'S PROBLEM SOLVING TO THE TYPES OF PROBLEMS YOU'LL SEE ON THE NEXT MILESTONE'S SET OF PROBLEMS?

For the next milestone, I think the current agent will still transfer quite well to problems that involve clean transformations and consistent object roles and colors. The current design has a good foundation for problems where there is a clear visual rule and where the training pairs constrain that rule in a straightforward way. I have confidence that this approach will scale well to the next milestone, even if the current rule bank needs to expand. That being said, I expect the next set of problems to place more emphasis on generalization. If the new problems involve more relationships or compositions, then the current system might struggle a bit.

My plan will be to generalize in two directions. First, I want to broaden the types of transformations that the agent can represent by allowing short sequences of compatible rules. Second, I want to improve the way it describes the grid, so that it can reason not only about raw layouts and colors, but about object roles, structure, and organization. My expectation is that this will matter much more in later milestones than it does in the current one.

8 WHAT FEEDBACK WOULD YOU HOPE TO GET FROM CLASSMATES ABOUT HOW YOUR AGENT COULD DO BETTER?

The most useful feedback would be idea for how to generalize better without making the rule bank too large. Right now, the design works because the search space is small and rules interpretable. The challenge is adding flexibility without

turning the agent into a gigantic collection of rules. An interesting design point I'd like to hear is when make a new rule versus when to expand a current one such that it covers more cases.

I'm also interested in failure analysis. One thing that is important to understand is whether a failure was because the rule bank needs something more, the right candidate was never generated, or if multiple rules fit the training data and the wrong generalized.

I'm almost interested in other approaches that didn't involve all brute-force methods such as computer vision and machine learning approaches.